New Horizons
Centro de Capacitación

# Práctica: Endpoints de Transición: Enviar y Revisar Solicitudes

*Paso a paso — Conectar la máquina de estados con endpoints HTTP y validar roles de usuario*

| | |
|---|---|
| **Curso** | Técnico de Desarrollo Full Stack con C# y Angular |
| **Sesión** | Sesión 6 — APIs del flujo de aprobación |
| **Docente** | Edwin Barrientos |
| **Duración estimada** | 60–90 minutos |
| **Nivel** | Básico (guía principal de la sesión) |
| **Entorno** | .NET 8 SDK · Visual Studio 2022 Community o VS Code · PostgreSQL o SQL Server Express |

## 1. Objetivos

- Construir el endpoint POST /api/solicitudes/{id}/enviar para cambiar el estado de Borrador a Enviada.
- Validar en el servicio y dominio que la solicitud contenga detalles y monto válido antes del envío.
- Construir el endpoint POST /api/solicitudes/{id}/revisar para el rol Revisor.
- Permitir que el Revisor observe y devuelva la solicitud a Borrador si la documentación está incompleta.
- Registrar cada acción en el historial inmutable de la entidad Aprobacion.

## 2. Desarrollo paso a paso

### Paso 1 — DTOs de flujo para Enviar y Revisar

En Application/Dtos/FlujoDtos.cs definimos los contratos para las transiciones.

**FlujoDtos.cs**

```
namespace SolicitudesApp.Application.Dtos;

public record RevisarSolicitudDto(int RevisorId, bool Conforme, string? Observaciones);
public record ResultadoTransicionDto(int SolicitudId, string EstadoResultante, string Mensaje);
```

### Paso 2 — Método de negocio Enviar a Revisión

En SolicitudService.cs agregamos la lógica de validación y transición a Enviada.

**SolicitudService.cs**

```
public async Task<ResultadoTransicionDto> EnviarAsync(int solicitudId)
{
    var solicitud = await _uow.Solicitudes.ObtenerConDetallesAsync(solicitudId);
    if (solicitud is null)
        throw new KeyNotFoundException($"No se encontró la solicitud con ID {solicitudId}.");

    // Ejecuta la máquina de estados de la entidad
    solicitud.EnviarARevision();

    await _uow.CompletarAsync();

    return new ResultadoTransicionDto(
        solicitud.Id,
        solicitud.Estado.ToString(),
        "La solicitud fue enviada a revisión exitosamente."
    );
}
```

## Paso 3 — Método de negocio Revisar Solicitud

Implementamos la revisión de sustentos permitiendo pasar a EnRevisión o devolver a Borrador con observaciones.

**SolicitudService.cs**

```
public async Task<ResultadoTransicionDto> RevisarAsync(int solicitudId, RevisarSolicitudDto dto)
{
    var solicitud = await _uow.Solicitudes.ObtenerConDetallesAsync(solicitudId);
    if (solicitud is null)
        throw new KeyNotFoundException($"No se encontró la solicitud con ID {solicitudId}.");

    if (dto.Conforme)
    {
        solicitud.MarcarEnRevision(dto.RevisorId, dto.Observaciones ?? "Documentación
conforme.");
    }
    else
    {
        if (string.IsNullOrWhiteSpace(dto.Observaciones))
            throw new ArgumentException("Debe indicar las observaciones para devolver la
solicitud.");

        solicitud.ObservarYDevolver(dto.RevisorId, dto.Observaciones);
    }

    await _uow.CompletarAsync();

    return new ResultadoTransicionDto(
        solicitud.Id,
        solicitud.Estado.ToString(),
        dto.Conforme ? "Solicitud en revisión." : "Solicitud devuelta a borrador por
observaciones."
    );
}
```

## Paso 4 — Exponer endpoints en SolicitudesController

Añadimos las rutas HTTP POST /api/solicitudes/{id}/enviar y /revisar.

SolicitudesController.cs

```
[HttpPost("{id:int}/enviar")]
public async Task<ActionResult<ResultadoTransicionDto>> Enviar(int id)
{
    var resultado = await _srv.EnviarAsync(id);
    return Ok(resultado);
}

[HttpPost("{id:int}/revisar")]
public async Task<ActionResult<ResultadoTransicionDto>> Revisar(int id, RevisarSolicitudDto dto)
{
    var resultado = await _srv.RevisarAsync(id, dto);
    return Ok(resultado);
}
```

### Paso 5 — Probar las transiciones con HTTP

Verificamos que el envío cambia el estado y que una observación devuelve la solicitud a Borrador.

flujo.http

```
### 1. Enviar solicitud a revisión
POST https://localhost:5001/api/solicitudes/1/enviar

### 2. Observar y devolver a borrador
POST https://localhost:5001/api/solicitudes/1/revisar
Content-Type: application/json

{
  "revisorId": 2,
  "conforme": false,
  "observaciones": "Falta adjuntar la cotización del segundo proveedor."
}
```

## 3. Retos adicionales

1. Impide que una solicitud en estado Borrador pueda ser revisada directamente sin haber sido enviada antes.
2. Verifica que si la solicitud no tiene ningún detalle de gasto, el método Enviar lance ReglaDominioException.
3. Escribe una prueba unitaria para validar que el estado vuelve a Borrador cuando conforme = false.
4. Registra en el log de consola la fecha y el usuario cada vez que se ejecute una revisión.

## 4. Entregables

- Endpoints /enviar y /revisar funcionando y retornando respuestas HTTP 200 con el estado resultante.
- Validación que impide enviar solicitudes sin detalles de gasto.

- Historial de aprobaciones registrando la observación del revisor en la base de datos.
- Pruebas automáticas demostrando el camino conforme y el camino observado.